Zachodniopomorski
Uniwersytet Technologiczny
w Szczecinie

Wydział Informatyki
kierunek studiów: Informatyka
specjalność: Inżynieria Oprogramowania

Praca dyplomowa inżynierska

SYSTEM WSPOMAGANIA ZARZĄDZANIA GOSPODARSTWEM DOMOWYM

HOUSEHOLD MANAGEMENT SUPPORT SYSTEM

Imię i Nazwisko

nr albumu: **XXXXXX**

Opiekun:

dr inż. Imię Nazwisko

Katedra Inżynierii Oprogramowania

Szczecin, 2026

Streszczenie

Przedmiotem niniejszej pracy jest projekt i implementacja systemu informatycznego „Household Manager” wspomagającego zarządzanie gospodarstwem domowym. Aplikacja rozwiązuje problemy związane z podziałem obowiązków, rozliczaniem wspólnych wydatków oraz komunikacją między domownikami.

System składa się z aplikacji mobilnej stworzonej w technologii React Native oraz serwera backendowego napisanego w języku Go, wykorzystującego bazę danych PostgreSQL. Kluczowe funkcjonalności obejmują: zarządzanie wirtualnymi domami, system przydzielania zadań, moduł finansowy z automatycznym odczytywaniem danych z rachunków (OCR), wspólne listy zakupów oraz ankiety decyzyjne. Zastosowanie nowoczesnych technologii pozwoliło na stworzenie wydajnego, skalowalnego i intuicyjnego w obsłudze rozwiązania, dostępnego na platformy Android i iOS.

słowa kluczowe: zarządzanie domem, aplikacja mobilna, React Native, Go, OCR, finanse osobiste

Abstract

The subject of this thesis is the design and implementation of the "Household Manager"IT system aimed at supporting household management. The application addresses issues related to the division of chores, settlement of shared expenses, and communication among household members.

The system consists of a mobile application built with React Native technology and a backend server written in the Go language, utilizing a PostgreSQL database. Key functionalities include: management of virtual homes, a task assignment system, a financial module with automatic data extraction from bills (OCR), shared shopping lists, and decision-making polls. The use of modern technologies allowed for the creation of an efficient, scalable, and intuitive solution, available for Android and iOS platforms.

keywords: household management, mobile application, React Native, Go, OCR, personal finance

Spis treści

Wstęp

Współczesne tempo życia oraz rosnąca liczba obowiązków sprawiają, że efektywne zarządzanie gospodarstwem domowym staje się wyzwaniem, szczególnie w przypadku osób dzielących przestrzeń życiową, takich jak studenci, współlokatorzy czy rodziny. Codzienne zadania, takie jak sprzątanie, zakupy, płacenie rachunków czy podejmowanie wspólnych decyzji, często stają się źródłem nieporozumień i konfliktów, gdy brakuje jasnego podziału odpowiedzialności i sprawnej komunikacji.

Tradycyjne metody organizacji, takie jak kartki na lodówce, ustne ustalenia czy chaotyczne konwersacje w komunikatorach internetowych, okazują się niewystarczające w obliczu złożoności współczesnego życia. Brak centralnego systemu do śledzenia wydatków, terminów płatności czy postępów w realizacji zadań domowych prowadzi do zapominania o ważnych terminach, dublowania zakupów oraz poczucia niesprawiedliwości w podziale obowiązków.

Celem niniejszej pracy jest zaprojektowanie i zaimplementowanie kompleksowego systemu informatycznego wspomagającego zarządzanie gospodarstwem domowym. Aplikacja ma na celu ułatwienie komunikacji między domownikami, automatyzację procesów związanych z płatnościami i zakupami oraz zapewnienie przejrzystości w podziale obowiązków.

Głównym elementem systemu jest aplikacja mobilna, która umożliwia użytkownikom tworzenie wirtualnych domów, zapraszanie do nich współlokatorów oraz wspólne zarządzanie zasobami. Kluczowe funkcjonalności obejmują system zarządzania zadaniami z możliwością przypisywania ich do konkretnych osób, moduł finansowy do śledzenia rachunków i wydatków (wspierany technologią OCR do automatycznego odczytu danych z faktur), wspólne listy zakupów oraz system ankiet do podejmowania demokratycznych decyzji.

Praca składa się z sześciu rozdziałów. Rozdział pierwszy zawiera analizę problemu oraz przegląd istniejących rozwiązań. Rozdział drugi przedstawia specyfikację wymagań funkcjonalnych i niefunkcjonalnych oraz projekt systemu. Rozdział trzeci opisuje wybrane technologie wykorzystane do realizacji projektu, w tym język Go dla warstwy serwerowej oraz framework React Native dla aplikacji mobilnej. Rozdział czwarty poświęcony jest szczegółom implementacyjnym, w tym strukturze bazy danych, architekturze API oraz rozwiązaniom z zakresu bezpieczeństwa. Rozdział piąty prezentuje wyniki testów i badań nad wytworzonym oprogramowaniem. Całość kończy podsumowanie i wnioski z realizacji projektu.

1. Analiza problemu i cel projektu

Wspólne mieszkanie, czy to w gronie rodziny, czy ze współlokatorami, wiąże się z koniecznością koordynacji wielu aspektów życia codziennego. Rozdział ten poświęcony jest analizie problemów związanych z zarządzaniem gospodarstwem domowym, zdefiniowaniu celu projektowanej aplikacji oraz przeglądowi istniejących na rynku rozwiązań.

1.1 Opis problemu współdzielenia obowiązków domowych

Zarządzanie domem to złożony proces, na który składa się szereg powtarzalnych czynności oraz decyzji podejmowanych przez grupę osób. Do najczęstszych problemów, z jakimi borykają się domownicy, należą:

- **Niejasny podział obowiązków:** Brak precyzyjnego przypisania zadań (np. sprzątanie łazienki, wynoszenie śmieci) prowadzi do sytuacji, w których obowiązki są zaniedbywane lub wykonywane ciągle przez te same osoby, co rodzi frustrację i konflikty.
- **Problemy z rozliczeniami finansowymi:** Wspólne wydatki na czynsz, media czy zakupy spożywcze wymagają sprawiedliwego podziału kosztów. Ręczne spisywanie wydatków jest uciążliwe i podatne na błędy, a brak terminowych spłat między współlokatorami zaburza płynność finansową.
- **Dublowanie zakupów lub braki w zapasach:** Brak zsynchronizowanej listy zakupów skutkuje kupowaniem niepotrzebnych produktów lub – przeciwnie – zapominaniem o artykułach pierwszej potrzeby.
- **Trudności w komunikacji i podejmowaniu decyzji:** Ustalenia dotyczące wspólnych spraw (np. wybór dostawcy internetu, planowanie posiłków) często giną w natłoku wiadomości na komunikatorach, co utrudnia osiągnięcie konsensusu.

Brak zorganizowanego systemu wsparcia tych procesów obciąża domowników tzw. „obciążeniem mentalnym” (ang. *mental load*), polegającym na konieczności ciągłego pamiętania o terminach i zobowiązaniach.

1.2 Cel i zakres aplikacji

Głównym celem projektu jest stworzenie aplikacji mobilnej „Household Manager”, która zintegruje kluczowe funkcjonalności niezbędne do sprawnego zarządzania domem w jednym, intuicyjnym interfejsie. System ma za zadanie zminimalizować ryzyko konfliktów poprzez zapewnienie transparentności działań i automatyzację powtarzalnych czynności.

Zakres funkcjonalny aplikacji obejmuje:

1. **Zarządzanie domami i użytkownikami:** Możliwość tworzenia wielu wirtualnych „domów”, zapraszania użytkowników poprzez kody lub linki oraz zarządzanie ich uprawnieniami.
2. **System zadań (Tasks):** Tworzenie, przypisywanie i śledzenie statusu prac domowych.
3. **Moduł finansowy (Bills):** Rejestrowanie rachunków, podział kosztów (równy lub według wag), śledzenie statusu płatności. Istotnym elementem jest integracja z mechanizmem OCR (Optical Character Recognition) pozwalającym na automatyczne wprowadzanie danych z fizycznych rachunków i faktur.
4. **Listy zakupów (Shopping Lists):** Współdzielone listy produktów z aktualizacją w czasie rzeczywistym, co pozwala na dodawanie i odhaczanie pozycji przez wszystkich domowników.
5. **System ankiet (Polls):** Narzędzie do demokratycznego podejmowania decyzji grupowych.
6. **Powiadomienia:** System alertów przypominających o nadchodzących terminach płatności czy przydzielonych zadaniach.

Aplikacja jest kierowana głównie do osób młodych, studentów wynajmujących wspólnie mieszkania oraz nowoczesnych rodzin, dla których smartfon jest naturalnym narzędziem organizacji życia.

1.3 Analiza istniejących rozwiązań

Na rynku dostępnych jest wiele narzędzi wspierających organizację, jednak większość z nich skupia się na wąskich dziedzinach lub jest adaptacją narzędzi biznesowych.

1.3.1 Splitwise

Splitwise to popularna aplikacja do dzielenia wydatków. Doskonale radzi sobie z matematyką podziału kosztów i upraszczaniem długów.

- **Zalety:** Rozbudowane opcje podziału kosztów, obsługa wielu walut.
- **Wady:** Brak funkcji zarządzania zadaniami domowymi, listami zakupów czy komunikacji grupowej niezwiązanej z finansami.

1.3.2 Trello / Asana

Narzędzia typu Kanban, popularne w zarządzaniu projektami w IT.

- **Zalety:** Elastyczność, możliwość tworzenia dowolnych tablic i list zadań.
- **Wady:** Zbyt skomplikowane do prostych zastosowań domowych, brak dedykowanych funkcji finansowych (np. podział rachunków), interfejs nastawiony na workflow biznesowy.

1.3.3 OurGroceries

Aplikacja dedykowana listom zakupów.

- **Zalety:** Prosta obsługa, synchronizacja w czasie rzeczywistym.
- **Wady:** Ograniczona tylko do zakupów, przestarzały interfejs użytkownika (UI), brak integracji z finansami i zadaniami.

1.3.4 Rozwiązania zintegrowane (np. OurHome, Cozi)

Istnieją aplikacje próbujące łączyć te funkcje, jednak często borykają się z problemami:

- Część z nich jest płatna w modelu subskrypcyjnym, co zniechęca studentów.
- Wiele projektów nie jest już rozwijanych i posiada przestarzały design.
- Rzadko oferują zaawansowane funkcje jak skanowanie rachunków (OCR).

1.4 Podsumowanie analizy

Analiza rynku wskazuje na brak darmowego, nowoczesnego i kompleksowego rozwiązania, które łączyłoby zarządzanie finansami (z wykorzystaniem OCR), zadaniami i zakupami w jednej aplikacji o wysokiej jakości interfejsu (UX/UI). Projektowany system „Household Manager” ma na celu wypełnienie tej luki, oferując użytkownikom narzędzie typu „wszystko w jednym”, dostosowane do współczesnych standardów aplikacji mobilnych.

2. Specyfikacja wymagań i projekt systemu

Rozdział ten definiuje wymagania stawiane systemowi „Household Manager” oraz przedstawia jego projekt architektoniczny i bazodanowy.

2.1 Wymagania funkcjonalne

Wymagania funkcjonalne określają, co system powinien robić. Zostały one podzielone na moduły odpowiadające głównym obszarom działania aplikacji.

2.1.1 Moduł Użytkownika i Uwierzytelniania

- **Rejestracja i logowanie:** Użytkownik może założyć konto podając adres e-mail i hasło lub skorzystać z logowania przez Google (OAuth 2.0). System weryfikuje poprawność wprowadzonych danych (np. format adresu e-mail, złożoność hasła).
- **Zarządzanie profilem:** Możliwość edycji awatara (zdjęcia profilowego) oraz danych osobowych (imię, nazwisko). Zmiana hasła wymaga ponownego uwierzytelnienia.
- **Przypomnienie hasła:** Procedura resetowania hasła drogą mailową, wykorzystująca unikalne tokeny jednorazowe.

2.1.2 Moduł Domu (Home)

- **Tworzenie domu:** Użytkownik może utworzyć nową przestrzeń (Dom), nadając jej nazwę. Automatycznie staje się administratorem nowo utworzonego domu.
- **Zarządzanie członkami:** Możliwość zapraszania nowych osób poprzez unikalny, 6-cyfrowy kod zaproszenia, ważny przez określony czas. Administrator może usuwać członków z domu.
- **Wiele domów:** Architektura systemu pozwala użytkownikowi na przynależność do wielu domów jednocześnie (np. "Mieszkanie Studenckie"i "Dom Rodzinny") oraz płynne przełączanie się między nimi w interfejsie aplikacji.

2.1.3 Moduł Zadań (Tasks)

- **CRUD Zadań:** Pełna obsługa cyklu życia zadania: tworzenie, edycja, usuwanie i wyświetlanie listy zadań. Każde zadanie posiada tytuł, opis oraz priorytet.
- **Przydzielanie:** Możliwość przypisania zadania do jednego lub wielu domowników. System obsługuje również zadania rotacyjne (cykliczne).
- **Terminy:** Definiowanie daty wykonania zadania (deadline). Aplikacja wyróżnia zadania przeterminowane.

- **Statusy:** Oznaczanie zadań jako wykonane, co jest widoczne dla wszystkich członków domu w czasie rzeczywistym.

2.1.4 Moduł Finansowy (Bills)

- **Dodawanie rachunków:** Wprowadzanie kwoty, tytułu, kategorii (np. media, czynsz, zakupy) oraz terminu płatności.
- **OCR (Optical Character Recognition):** Innowacyjna funkcja pozwalająca na zrobienie zdjęcia fizycznego paragonu lub faktury. System automatycznie przetwarza obraz, ekstrahując kwotę całkowitą i nazwę sprzedawcy, co znacząco przyspiesza proces wprowadzania danych.
- **Podział kosztów:** Elastyczny podział kwoty rachunku na członków domu. Domyślnie system proponuje podział równomierny, ale użytkownik może go modyfikować.
- **Rozliczanie:** Śledzenie statusu płatności (kto już zapłacił swoją część, a kto zalega).

2.1.5 Moduł Zakupów (Shopping)

- **Lista zakupów:** Dodawanie produktów do wspólnej listy zakupowej.
- **Kategorie:** Grupowanie produktów (np. spożywcze, chemia, warzywa) w celu łatwiejszej nawigacji w sklepie.
- **Interakcja:** Odhaczanie kupionych produktów, co skutkuje ich przeniesieniem do sekcji "Kupionełub usunięciem z listy. Synchronizacja stanu listy między urządzeniami zapobiega dublowaniu zakupów.

2.1.6 Moduł Ankiet (Polls)

- **Tworzenie ankiet:** Możliwość zadawania pytań dotyczących wspólnych spraw (np. Ćo na obiad?", "Jakiego dostawcę internetu wybieramy?") i definiowania opcji odpowiedzi.
- **Głosowanie:** Anonimowe lub jawne oddawanie głosu przez domowników.
- **Wyniki:** Wizualizacja wyników ankiety w formie wykresu lub listy procentowej.

2.2 Wymagania niefunkcjonalne

- **Dostępność (Availability):** System powinien być dostępny 24 godziny na dobę, 7 dni w tygodniu, z planowanymi oknami serwisowymi w godzinach nocnych.
- **Skalowalność:** Architektura backendu musi umożliwiać łatwe skalowanie poziome (dodawanie kolejnych instancji serwera) w przypadku wzrostu liczby użytkowników, co realizowane jest poprzez konteneryzację (Docker).
- **Bezpieczeństwo:**
  - Hasła użytkowników muszą być przechowywane w formie zahaszowanej przy użyciu silnego algorytmu (np. bcrypt).
  - Komunikacja między klientem a serwerem musi być szyfrowana protokołem TLS/SSL (HTTPS).
  - Dostęp do zasobów API jest zabezpieczony mechanizmem tokenów JWT (JSON Web Tokens).

- **Responsywność i wydajność:** Aplikacja mobilna powinna działać płynnie na systemach Android (wersja 10+) i iOS (wersja 15+). Czas odpowiedzi API nie powinien przekraczać 200ms dla 95% standardowych zapytań (np. pobranie listy zadań).
- **Internacjonalizacja:** System powinien być przygotowany do obsługi wielu wersji językowych (i18n), z domyślnym wsparciem dla języka polskiego i angielskiego.

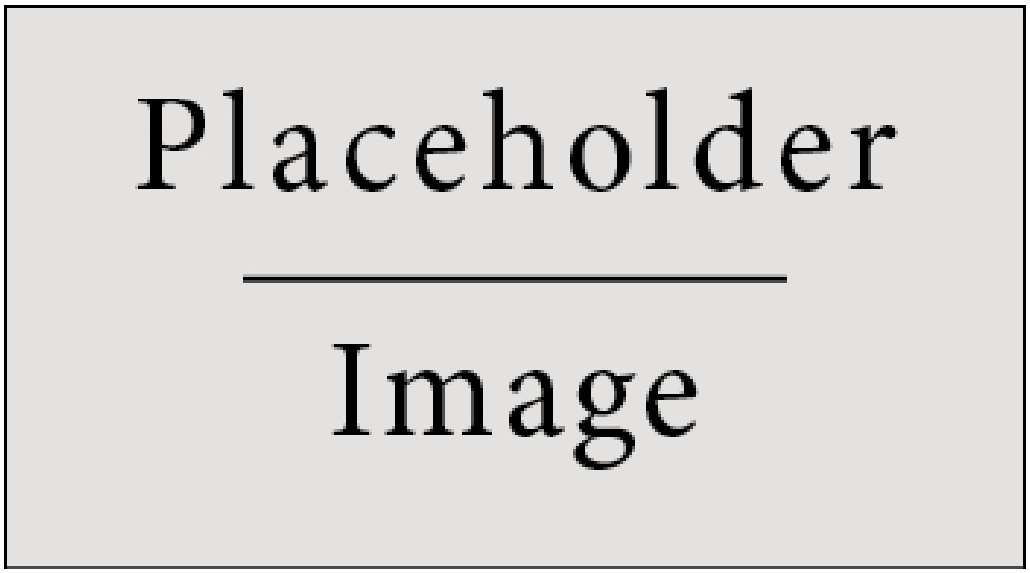
2.3 Projekt systemu

2.3.1 Architektura klient-serwer

System „Household Manager” został zaprojektowany w oparciu o klasyczną, trójwarstwową architekturę klient-serwer (ang. *Three-Tier Architecture*), która zapewnia logiczną separację warstwy prezentacji, logiki biznesowej oraz danych. Takie podejście ułatwia niezależny rozwój poszczególnych komponentów oraz ich utrzymanie.

- **Warstwa Prezentacji (Klient):** Aplikacja mobilna zaimplementowana w frameworku React Native. Odpowiada za interakcję z użytkownikiem, wyświetlanie danych oraz przesyłanie żądań do serwera. Klient jest ćienki"(ang. *thin client*), co oznacza, że większość ciężkiej logiki biznesowej została przeniesiona na serwer.
- **Warstwa Logiki Biznesowej (Serwer):** Serwer REST API napisany w języku Go. Przetwarza żądania przychodzące z aplikacji mobilnej, wykonuje operacje na danych, autoryzuje użytkowników i komunikuje się z bazą danych.
- **Warstwa Danych (Baza Danych):** Relacyjna baza danych PostgreSQL przechowująca wszystkie trwałe informacje systemu.

Poniższy schemat (Rysunek **??**) prezentuje ogólną architekturę systemu oraz przepływ danych między komponentami.



Rysunek 2.1: Schemat architektury systemu Household Manager

2.3.2 Projekt bazy danych PostgreSQL

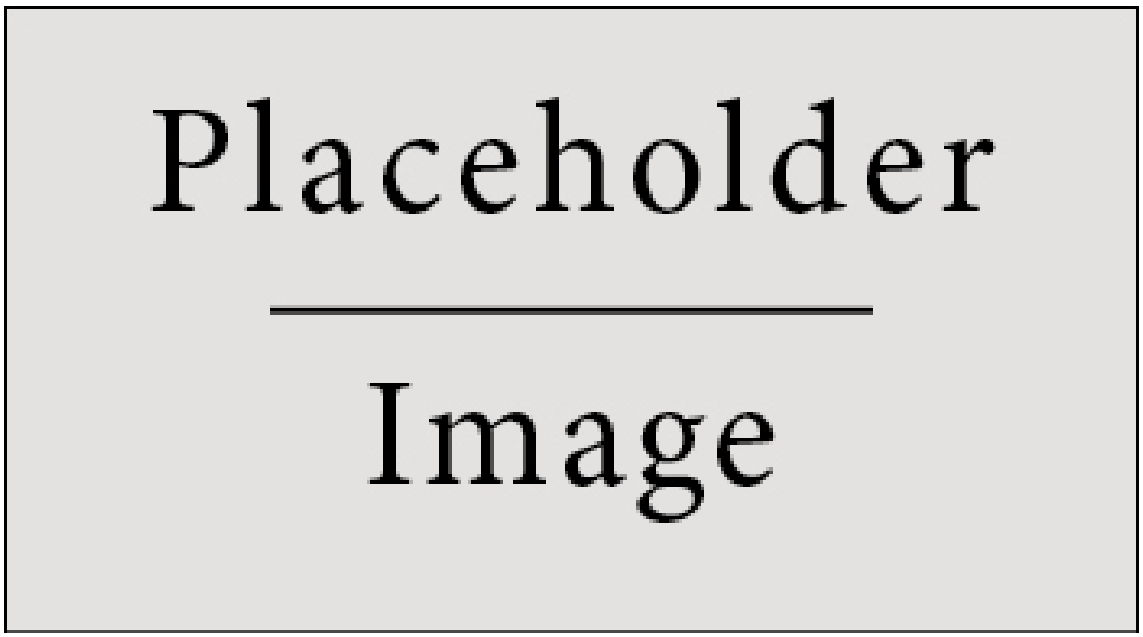
Jako system zarządzania bazą danych (RDBMS) wybrano **PostgreSQL**. Decyzja ta podyktowana była kilkoma kluczowymi czynnikami:

1. **Integralność danych (ACID):** PostgreSQL jest w pełni zgodny ze standardem ACID (Atomicity, Consistency, Isolation, Durability), co jest kluczowe w systemach finansowych (moduł rachunków), gdzie nie można pozwolić sobie na utratę lub niespójność danych transakcyjnych.
2. **Wsparcie dla JSONB:** Baza ta oferuje doskonałą obsługę typu danych JSON, co pozwala na elastyczne przechowywanie mniej ustrukturyzowanych danych (np. metadane logów OCR) obok tradycyjnych tabel relacyjnych.
3. **Wydajność i rozszerzalność:** PostgreSQL słynie z wysokiej wydajności przy złożonych zapytaniach oraz możliwości tworzenia własnych indeksów i funkcji.

2.3.3 Diagram ERD

Struktura bazy danych została zaprojektowana tak, aby odzwierciedlać rzeczywiste relacje panujące w gospodarstwie domowym. Centralnym punktem schematu są tabele użytkowników (`users`) oraz domów (`homes`), połączone relacją wiele-do-wielu poprzez tabelę łącznikową (`home_memberships`).

Poniższy diagram ERD (Entity Relationship Diagram) przedstawia szczegółowy schemat bazy danych.



Rysunek 2.2: Diagram ERD bazy danych

Kluczowe encje w systemie to:

- **users:** Przechowuje unikalne dane identyfikacyjne użytkowników (adres email, hash hasła) oraz dane profilowe.
- **homes:** Reprezentuje pojedyncze gospodarstwo domowe. Posiada unikalny identyfikator oraz kod zaproszenia.

- **home_memberships:** Tabela asocjacyjna definiująca przynależność użytkownika do domu oraz jego rolę (np. ADMIN, MEMBER).
- **tasks:** Tabela zadań zawierająca tytuł, opis, datę wykonania oraz status.
- **task_assignments:** Definiuje, który użytkownik jest odpowiedzialny za dane zadanie. Umożliwia przypisanie wielu osób do jednego zadania.
- **bills:** Przechowuje informacje o rachunkach: kwotę całkowitą, datę płatności, wystawcę rachunku oraz status (opłacony/nieopłacony).
- **shopping_items** oraz **shopping_categories:** Tabele obsługujące moduł zakupów, pozwalające na kategoryzację produktów.

2.3.4 Modele GORM i relacje

W warstwie aplikacji (backend w Go) wykorzystano bibliotekę **GORM** (Go Object Relational Mapping). Jest to potężne narzędzie mapowania obiektowo-relacyjnego, które pozwala programiście operować na strukturach języka Go (structs) zamiast pisać surowe zapytania SQL.

Zalety wykorzystania GORM w projekcie:

- **Bezpieczeństwo:** Automatyczna ochrona przed atakami typu SQL Injection dzięki parametryzacji zapytań.
- **Automigracje:** GORM potrafi automatycznie tworzyć i aktualizować schemat bazy danych na podstawie definicji struktur w kodzie (Code-First approach), co znacznie przyspiesza prototypowanie.
- **Abstrakcja:** Pozwala na łatwą zmianę silnika bazy danych w przyszłości (np. na MySQL czy SQLite) bez konieczności przepisywania logiki aplikacji.

Wadą rozwiązań typu ORM może być narzut wydajnościowy przy bardzo skomplikowanych zapytaniach analitycznych, jednak w przypadku typowych operacji CRUD (Create, Read, Update, Delete) realizowanych w systemie Household Manager, korzyści wynikające z szybkości wytwarzania oprogramowania przeważają nad minimalnym spadkiem wydajności.

3. Wybór technologii

Wybór odpowiednich technologii jest kluczowym etapem procesu wytwarzania oprogramowania, mającym bezpośredni wpływ na wydajność, skalowalność, bezpieczeństwo oraz łatwość późniejszego utrzymania systemu (maintainability). Decyzje podjęte na tym etapie determinują szybkość procesu deweloperskiego oraz komfort pracy programistów. W niniejszym rozdziale przedstawiono szczegółową charakterystykę stosu technologicznego (tech stack) wykorzystanego do realizacji projektu „Household Manager".

3.1 Backend — Go, GORM, PostgreSQL, Chi Router

Warstwa serwerowa (backend) stanowi mózg aplikacji, odpowiadający za logikę biznesową, przetwarzanie danych oraz komunikację z bazą danych. Do jej implementacji wybrano język **Go** (znany również jako Golang).

3.1.1 Uzasadnienie wyboru Go

Język Go został stworzony w firmie Google w 2007 roku przez Roberta Griesemera, Roba Pike'a i Kena Thompsona. Jego pierwsza stabilna wersja ukazała się w 2012 roku. Go powstał jako odpowiedź na problemy związane z zarządzaniem dużymi bazami kodu, długim czasem kompilacji oraz trudnościami w pisaniu wydajnego kodu współbieżnego w starszych językach, takich jak C++ czy Java.

Główne powody wyboru Go do tego projektu to:

1. **Wydajność kompilowana:** Go jest językiem kompilowanym bezpośrednio do kodu maszynowego, co sprawia, że aplikacje napisane w Go są znacznie szybsze niż te uruchamiane w środowiskach interpretowanych (jak Python czy JavaScript/Node.js). Nie wymaga maszyny wirtualnej (JVM), co przekłada się na mniejsze zużycie pamięci RAM i szybszy start aplikacji.
2. **Statyczne typowanie:** Silny system typów pomaga wykrywać błędy już na etapie kompilacji, zanim kod trafi na produkcję. W przeciwieństwie do języków dynamicznych, programista ma pewność, że zmienna zadeklarowana jako liczba całkowita zawsze nią będzie, co zwiększa stabilność systemu.
3. **Współbieżność (Concurrency):** Go posiada unikalny model współbieżności oparty na tzw. *goroutines* i kanałach (*channels*). Goroutine to łekki wątek", zarządzany przez runtime języka, a nie przez system operacyjny. Uruchomienie tysięcy goroutines wymaga ułamka zasobów potrzebnych dla tradycyjnych wątków systemowych. W kontekście serwera HTTP oznacza to możliwość jednoczesnej obsługi ogromnej liczby zapytań od użytkowników przy minimalnym narzucie sprzętowym.

4. **Prostota i czytelność:** Składnia Go jest minimalistyczna (brak klas, dziedziczenia w tradycyjnym sensie, skomplikowanych meta-programowania). Wymusza to pisanie prostego, czytelnego kodu, który jest łatwy do zrozumienia dla nowych członków zespołu. Sformatowany kod narzędziem `gofmt` wygląda zawsze tak samo, co eliminuje spory o styl kodowania.

W porównaniu do popularnego Node.js, Go oferuje lepszą wydajność obliczeniową (istotne np. przy przetwarzaniu obrazów w module OCR) oraz stabilniejszy ekosystem bibliotek standardowych.

3.1.2 Framework i biblioteki backendowe

W ekosystemie Go panuje tendencja do unikania "kobyłowatych"frameworków typu "wszystko w jednym"(jak Django w Pythonie czy Spring w Javie) na rzecz komponowania aplikacji z mniejszych, wyspecjalizowanych bibliotek. W projekcie wykorzystano:

- **Chi Router:** Chi to lekki, szybki i idiomatyczny router HTTP dla Go. Jego główną zaletą jest pełna kompatybilność ze standardowym interfejsem `net/http`. Chi nie narzuca struktury aplikacji, ale oferuje potężny system *middleware* (oprogramowania pośredniczącego). Middleware w Chi pozwala na łatwe ópinanie"handlerów dodatkową logiką, taką jak logowanie żądań, obsługa CORS, kompresja GZIP czy weryfikacja tokenów JWT, w sposób deklaratywny i czytelny.
- **GORM (Go Object Relational Mapping):** Jest to najpopularniejsza biblioteka ORM dla Go. Umożliwia ona pracę z bazą danych przy użyciu obiektów (struktur), a nie surowego SQL. GORM automatyzuje wiele powtarzalnych czynności: mapowanie wyników zapytań na struktury, obsługę transakcji, a także zarządzanie relacjami (one-to-one, one-to-many, many-to-many). Dzięki mechanizmowi *Hooks*, GORM pozwala również na wykonywanie akcji przed lub po zapisie danych (np. automatyczne haszowanie hasła przed zapisem użytkownika).
- **Gosseract:** Jest to wrapper (interfejs) dla języka Go do popularnego silnika OCR Tesseract, rozwijanego przez Google. Tesseract wykorzystuje sieci neuronowe (LSTM - Long Short-Term Memory) do rozpoznawania tekstu na obrazach. Biblioteka Gosseract umożliwia łatwe przekazanie obrazu (z pliku lub bufora pamięci) do silnika C++ Tesseracta i odebranie wynikowego tekstu, który następnie jest parsowany przez logikę aplikacji w celu znalezienia kwot i dat na rachunkach.

3.2 Frontend — React Native, Expo, TypeScript

Warstwa kliencka (aplikacja mobilna) została zrealizowana w technologii **React Native**, która umożliwia tworzenie natywnych aplikacji na systemy Android i iOS przy użyciu języka JavaScript/TypeScript i biblioteki React.

3.2.1 Charakterystyka React Native

React Native, stworzony przez Facebooka (Meta), różni się od rozwiązań hybrydowych (jak Ionic czy Cordova) tym, że nie renderuje interfejsu w komponencie WebView (przeglądarce internetowej). Zamiast tego, kod JavaScript komunikuje się z natywnym API platformy poprzez tzw. "Most"(Bridge) lub – w nowszej architekturze – poprzez JSI (JavaScript Interface). Dzięki temu, komponenty używane w kodzie (np. `<View>`, `<Text>`) są

tłumaczone na natywne widoki systemowe (`android.view.View`, `UIView`). Zapewnia to wydajność i odczucia z użytkowania (Look and Feel) zbliżone do aplikacji pisanych w Javie czy Swifcie, przy zachowaniu wspólnego kodu dla obu platform (tzw. *Code Sharing* na poziomie 90-95%).

Wykorzystanie składni JSX (JavaScript XML) pozwala na deklaratywne tworzenie interfejsu użytkownika, co znacznie ułatwia budowanie skomplikowanych widoków z mniejszych, reużywalnych komponentów.

3.2.2 Platforma Expo

Projekt wykorzystuje platformę **Expo** – zestaw narzędzi i usług zbudowanych wokół React Native. Decyzja o użyciu Expo zamiast ćzystegoŻeact Native CLI (Bare Workflow) wynika z kilku korzyści:

- **Szybki start i konfiguracja:** Expo eliminuje konieczność ręcznej konfiguracji środowiska Android Studio i Xcode na początku projektu.
- **Expo Go:** Możliwość natychmiastowego testowania aplikacji na fizycznym urządzeniu poprzez zeskanowanie kodu QR, bez konieczności kompilacji pliku .apk/.ipa przy każdej zmianie.
- **Dostęp do natywnego API:** Expo dostarcza gotowe, przetestowane moduły (SDK) do obsługi aparatu, systemu plików, powiadomień Push, lokalizacji czy sensorów, co przyspiesza development.
- **EAS (Expo Application Services):** Ułatwia proces budowania i publikowania aplikacji w sklepach Google Play i App Store w chmurze, bez obciążania lokalnego komputera.

3.2.3 Zarządzanie stanem i komunikacja z API

W nowoczesnych aplikacjach frontendowych kluczowe jest efektywne zarządzanie stanem (danymi, które zmieniają się w czasie).

- **Zustand:** Do zarządzania stanem globalnym wybrano bibliotekę Zustand. Jest ona znacznie lżejsza i prostsza w obsłudze niż popularny Redux. Redux wymaga dużej ilości kodu "boilerplate"(akcje, reducery, types, middleware), co w mniejszych i średnich projektach bywa uciążliwe. Zustand opiera się na prostych hookach i nie wymaga owijania całej aplikacji w *Provider*. Pozwala to na łatwy dostęp do danych (np. profilu zalogowanego użytkownika) z dowolnego miejsca w aplikacji.
- **Axios:** Do komunikacji z API RESTowym wykorzystano klienta Axios. Jest to biblioteka oparta na obietnicach (Promises), która automatycznie transformuje dane do formatu JSON, obsługuje nagłówki (np. dodawanie tokena autoryzacyjnego) oraz zapewnia lepszą obsługę błędów niż wbudowana w przeglądarki funkcja `fetch`.

3.3 Integracja backend–frontend (REST API)

Komunikacja między aplikacją mobilną a serwerem odbywa się zgodnie z zasadami architektury **REST** (Representational State Transfer). Główne założenia REST spełnione w projekcie:

- **Bezstanowość (Stateless):** Każde żądanie HTTP zawiera wszystkie informacje niezbędne do jego obsłużenia (np. token autoryzacyjny). Serwer nie przechowuje stanu sesji użytkownika między zapytaniami.
- **Jednolity interfejs:** Zasoby są identyfikowane przez adresy URL (np. `/api/homes`), a operacje na nich wykonywane są za pomocą standardowych metod HTTP (GET, POST, PUT, DELETE).
- **Format danych:** Wymiana danych odbywa się w lekkim formacie JSON, który jest natywnie wspierany zarówno przez JavaScript (Frontend), jak i biblioteki Go (Backend).

Wybrano REST zamiast GraphQL ze względu na prostotę implementacji, łatwość cachowania zapytań HTTP oraz jasny podział odpowiedzialności.

3.4 Testy jednostkowe i integracyjne w Go

Jakość kodu serwerowego jest weryfikowana za pomocą testów. Wykorzystano bibliotekę **Testify**, która rozszerza standardowy pakiet `testing` o asercje (np. `assert.Equal`, `assert.Nil`) oraz narzędzia do mockowania zależności. Testy obejmują kluczowe elementy logiki biznesowej oraz handlery HTTP.

3.5 Podsumowanie

Dobór technologii w projekcie „Household Manager” podyktowany był chęcią stworzenia nowoczesnego, skalowalnego i łatwego w utrzymaniu systemu. Połączenie wydajnego backendu w Go z elastycznym frontendem w React Native stanowi solidną podstawę dla aplikacji użytkowej.

4. Implementacja systemu

W tym rozdziale przedstawiono szczegóły implementacyjne systemu „Household Manager”, w tym strukturę kodu, konfigurację routingu, model danych, system autoryzacji oraz budowę aplikacji klienckiej. Zaprezentowano również kluczowe fragmenty kodu źródłowego oraz zrzuty ekranu gotowej aplikacji.

4.1 Struktura projektu backend

Aplikacja serwerowa została zaprojektowana zgodnie z zasadami architektury *Clean Architecture* (Czysta Architektura), co zapewnia separację warstw, niezależność od frameworków oraz łatwość testowania. Kod podzielony jest na wyraźne warstwy odpowiedzialności:

1. **Warstwa Transportowa (Handlers/Controllers):** Odpowiada za obsługę protokołu HTTP. Odbiera żądania, waliduje dane wejściowe (DTO), wywołuje odpowiednie serwisy i zwraca odpowiedź w formacie JSON. Nie zawiera logiki biznesowej.
2. **Warstwa Usług (Services):** Serce aplikacji. Tutaj znajduje się cała logika biznesowa (np. obliczanie podziału rachunku, generowanie kodu zaproszenia). Serwisy operują na modelach domenowych i korzystają z repozytoriów.
3. **Warstwa Dostępu do Danych (Repositories):** Odpowiada za bezpośrednią komunikację z bazą danych. Ukrywa szczegóły implementacyjne zapytań SQL/GORM przed resztą aplikacji.
4. **Modele (Models):** Struktury danych reprezentujące encje biznesowe (np. User, Bill).

Struktura katalogów projektu prezentuje się następująco:

- `cmd/server/main.go`: Punkt wejścia aplikacji, inicjalizacja konfiguracji, połączenia z DB i start serwera.
- `internal/config`: Obsługa zmiennych środowiskowych (ładowanie z pliku .env).
- `internal/http/handlers`: Kontrolery HTTP.
- `internal/http/middleware`: Oprogramowanie pośredniczące (logowanie, autoryzacja JWT, obsługa CORS).
- `internal/models`: Definicje struktur danych i tagów GORM.
- `internal/repository`: Warstwa DAO (Data Access Object).
- `internal/services`: Logika biznesowa.
- `internal/router`: Konfiguracja routera Chi i mapowanie ścieżek.

4.2 Endpoints API i routing w Chi

Do obsługi routingu wykorzystano bibliotekę `go-chi`. Definiuje ona drzewiastą strukturę ścieżek, co ułatwia zarządzanie dużą liczbą endpointów.

```
r.Route("/homes", func(r chi.Router) {
    r.Post("/create", homeHandler.Create) // Tworzenie domu
    r.Post("/join", homeHandler.Join)     // Dolaczanie do domu
    r.Get("/my", homeHandler.GetUserHome) // Pobieranie domu uzytkownika

    r.Route("/{home_id}", func(r chi.Router) {
        // Middleware sprawdzajacy czy uzytkownik jest czlonkiem domu
        r.With(middleware.RequireMember(homeRepo)).Get("/", homeHandler.GetByID)

        // Zagniezdzone zasoby: zadania, rachunki, zakupy
        r.Route("/tasks", func(r chi.Router) {
            r.Get("/", taskHandler.GetTasksByHomeID)
            r.Post("/", taskHandler.Create)
        })
        r.Route("/bills", func(r chi.Router) {
            r.Get("/", billHandler.GetByHomeID)
        })
    })
})
```

Algorytm 4.1: Konfiguracja routingu dla zasobów Home

4.2.1 Dokumentacja API (Swagger)

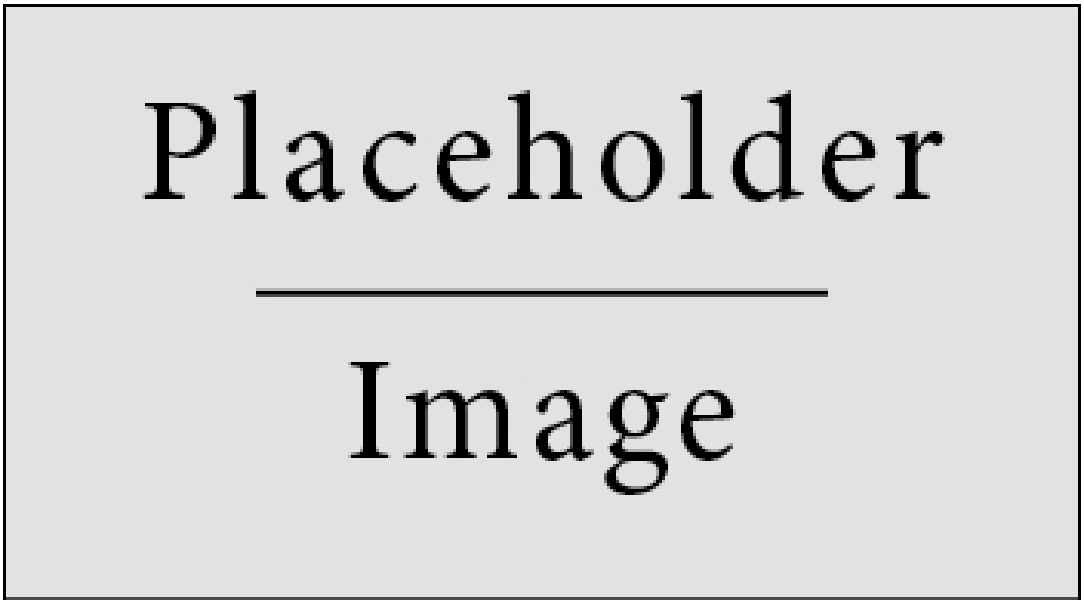
Wszystkie endpointy zostały udokumentowane przy użyciu standardu OpenAPI (Swagger). Poniższy zrzut ekranu przedstawia interaktywną dokumentację wygenerowaną automatycznie.

4.3 Modele danych i migracje

Baza danych jest zarządzana przez bibliotekę GORM. Struktury Go są mapowane na tabele w bazie danych. Wykorzystano tagi strukturalne (`‘gorm:"...„`) do definiowania ograniczeń (constraints), indeksów oraz relacji.

Przykładowa, rozbudowana struktura modelu rachunku (`Bill`):

```
type Bill struct {
    ID          uuid.UUID
        ‘gorm:"type:uuid;default:gen_random_uuid();primaryKey"‘
    Title       string     ‘gorm:"not null"‘
```



Rysunek 4.1: Dokumentacja API w Swagger UI

```
    Amount       float64    `gorm:"not null"`
    PayerID      uuid.UUID  `gorm:"type:uuid;not null"` // Klucz obcy do User
    Payer        User       `gorm:"foreignKey:PayerID"` // Relacja
    HomeID       uuid.UUID  `gorm:"type:uuid;not null"` // Klucz obcy do Home
    CategoryID   *uuid.UUID `gorm:"type:uuid"`
    Category     *BillCategory `gorm:"foreignKey:CategoryID"`
    DueDate      time.Time
    IsPaid       bool       `gorm:"default:false"`
    ReceiptURL   string     // URL do zdjecia w S3

    // Pola audytowe
    CreatedAt    time.Time
    UpdatedAt    time.Time
    DeletedAt    gorm.DeletedAt `gorm:"index"` // Obsluga Soft Delete
}
```

Algorytm 4.2: Model rachunku (Bill) w GORM

4.4 System autoryzacji i JWT

Bezpieczeństwo API opiera się na tokenach JWT (JSON Web Tokens). Proces autoryzacji przebiega następująco:

1. Użytkownik loguje się podając email i hasło.

2. Serwer weryfikuje hasło (porównuje hash bcrypt).
3. W przypadku sukcesu, generowany jest token JWT zawierający $\texttt{user}_{i}dorazdatwyganicia(np.72h).Tokenjes$

4.5 Obsługa błędów i walidacja danych

Wszystkie błędy zwracane przez API mają ujednoliconą strukturę JSON, co ułatwia ich obsługę po stronie klienta. Walidacja danych wejściowych odbywa się przy użyciu biblioteki `go-playground/validator`.

```
{
    "error": "Validation Failed",
    "details": {
        "Email": "field is required",
        "Password": "min length is 8"
    }
}
```

Algorytm 4.3: Struktura błędu API

4.6 Integracja z OCR dla rachunków

Moduł OCR (*Optical Character Recognition*) umożliwia automatyczne zaczytywanie danych z rachunków. Logika parsowania tekstu z paragonu opiera się na wyrażeniach regularnych (Regex), które poszukują wzorców dat i kwot.

Fragment serwisu odpowiedzialnego za analizę tekstu z paragonu:

```
func (s *OCRService) ParseReceiptText(text string)
    (float64, string) {
    // Regex dla kwot (np. 123.45, 123,45 PLN)
    amountRegex :=
        regexp.MustCompile(`(\d+[.,]\d{2})\s*(?:PLN|zl|EUR)?`)

    lines := strings.Split(text, "\n")
    var maxAmount float64
    var vendor string

    for i, line := range lines {
        if i == 0 {
            vendor = line // Zalozenie: pierwsza linia
                to nazwa sklepu
        }
        matches := amountRegex.FindStringSubmatch(line)
        if len(matches) > 1 {
            // Konwersja przecinka na kropke i
                parsowanie float
            val, err :=
                strconv.ParseFloat(strings.Replace(matches[1],
                ",", ".", 1), 64)
```

```
                if err == nil && val > maxAmount {
                    maxAmount = val // Szukamy najwiekszej
                        kwoty (Suma)
                }
            }
        }
        return maxAmount, vendor
}
```

Algorytm 4.4: Logika parsowania tekstu z paragonu

4.7 Struktura projektu frontend

Aplikacja kliencka w React Native wykorzystuje **Expo Router**, który opiera routing na strukturze plików (File-based routing), podobnie jak framework Next.js. Katalog `app` zawiera ekrany aplikacji:

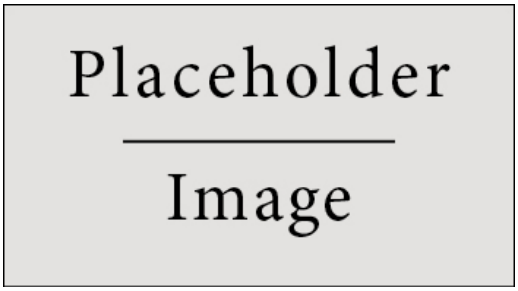
- `app/index.tsx`: Ekran startowy (Splash screen).
- `app/(auth)/login.tsx`: Ekran logowania (grupa auth, niedostępna po zalogowaniu).
- `app/(tabs)/`$layout.tsx : Konfiguracjadolnegopaskanawigacji(TabBar)$.`app/(tabs)/ho`
  $Gwnypulpituytkownika$.
- `app/[id].tsx`: Dynamiczna ścieżka (np. szczegóły zadania).

4.8 Interfejs użytkownika (UI) - Prezentacja aplikacji

Interfejs użytkownika został zaprojektowany z naciskiem na przejrzystość i łatwość obsługi (User Experience). Poniżej przedstawiono kluczowe ekrany aplikacji.

4.8.1 Ekran logowania i rejestracji

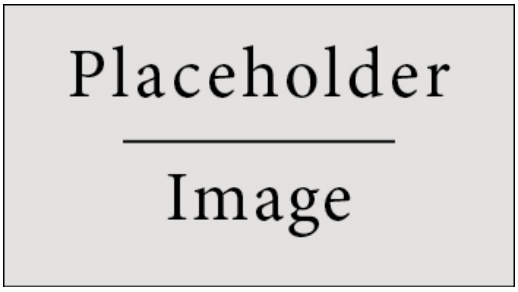
Umożliwia dostęp do aplikacji. Użytkownik może zalogować się tradycyjnie lub przez Google.



Rysunek 4.2: Ekran logowania

4.8.2 Pulpit główny (Dashboard)

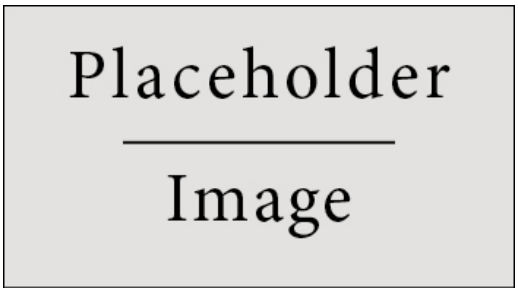
Centrum dowodzenia domem. Wyświetla podsumowanie: najbliższe zadania, stan konta oraz skróty do najważniejszych funkcji.



Rysunek 4.3: Pulpit główny aplikacji

4.8.3 Zarządzanie zadaniami

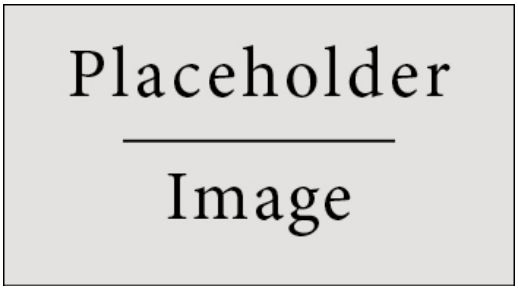
Lista zadań z możliwością filtrowania i oznaczania jako wykonane.



Rysunek 4.4: Lista zadań domowych

4.8.4 Skanowanie rachunków (OCR)

Moduł pozwalający na zrobienie zdjęcia i weryfikację odczytanych danych.



Rysunek 4.5: Ekran dodawania rachunku ze skanowaniem OCR

4.9 Wybrane fragmenty kodu frontendowego

Poniżej przedstawiono niestandardowy React Hook odpowiedzialny za pobieranie danych z API z obsługą stanu ładowania i błędów.

```
export function useFetch<T>(url: string) {
  const [data, setData] = useState<T | null>(null);
```

```
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const response = await api.get(url);
        setData(response.data);
      } catch (err) {
        setError(err);
      } finally {
        setLoading(false);
      }
    };
    fetchData();
  }, [url]);

  return { data, loading, error, refetch: fetchData };
}
```

Algorytm 4.5: Custom Hook useFetch

5. Badania i testy systemu

Celem tego rozdziału jest weryfikacja poprawności działania wytworzonego oprogramowania oraz ocena stopnia realizacji założonych wymagań. Przeprowadzono szereg testów funkcjonalnych oraz wydajnościowych.

5.1 Scenariusze testowe

W celu sprawdzenia kluczowych funkcjonalności systemu opracowano następujące scenariusze testowe, odzwierciedlające typowe ścieżki użytkownika:

5.1.1 Scenariusz 1: Konfiguracja nowego gospodarstwa domowego

1. Użytkownik A rejestruje nowe konto.
2. Użytkownik A tworzy nowy Dom o nazwie „Mieszkanie Studenckie”.
3. Użytkownik A generuje kod zaproszenia i przekazuje go Użytkownikowi B.
4. Użytkownik B rejestruje się i dołącza do domu używając otrzymanego kodu.
5. **Oczekiwany rezultat:** Obaj użytkownicy widzą ten sam pulpit domu i mają dostęp do wspólnych zasobów.

5.1.2 Scenariusz 2: Dodawanie i rozliczanie rachunku z OCR

1. Użytkownik wybiera opcję dodania rachunku i robi zdjęcie paragonu.
2. System przetwarza obraz i proponuje kwotę oraz nazwę sklepu.
3. Użytkownik zatwierdza dane i wybiera podział kosztów „po równo”.
4. Rachunek pojawia się na liście jako „nieopłacony”.
5. **Oczekiwany rezultat:** Kwota zostaje poprawnie odczytana, a dług zostaje przypisany do pozostałych domowników.

5.2 Testy funkcjonalne backendu

Testy jednostkowe i integracyjne warstwy serwerowej zostały zaimplementowane przy użyciu pakietu `testing` oraz biblioteki `testify`.

Przykładowy wynik uruchomienia testów dla serwisu autoryzacji:

```
$ go test ./internal/services/... -v
=== RUN   TestRegisterUser
--- PASS: TestRegisterUser (0.00s)
=== RUN   TestLoginUser_Success
--- PASS: TestLoginUser_Success (0.00s)
```

```
=== RUN   TestLoginUser_InvalidPassword
--- PASS: TestLoginUser_InvalidPassword (0.00s)
PASS
ok
    github.com/Dragodui/diploma-server/internal/services
        0.123s
```

Algorytm 5.1: Wynik testów jednostkowych

Wszystkie kluczowe ścieżki krytyczne (rejestracja, tworzenie domu, dodawanie zadań) zostały pokryte testami automatycznymi, co zapewnia stabilność API.

5.3 Testy interfejsu mobilnego

Testy aplikacji mobilnej przeprowadzono manualnie na dwóch urządzeniach fizycznych:

- Google Pixel 6 (Android 14)
- iPhone 13 (iOS 17)

oraz na emulatorach.

Weryfikowano:

- **Responsywność:** Czy elementy UI skalują się poprawnie na różnych rozdzielczościach.
- **Nawigację:** Czy przechodzenie między ekranami jest płynne i intuicyjne.
- **Obsługę błędów:** Czy aplikacja wyświetla czytelne komunikaty w przypadku braku internetu lub błędu serwera.

Stwierdzono, że aplikacja działa stabilnie na obu platformach. Wykryte drobne błędy wizualne (np. przesunięcia tekstu) zostały na bieżąco poprawione.

5.4 Wyniki i wnioski z testów

Przeprowadzone badania wykazały, że system spełnia założone wymagania funkcjonalne.

- **Wydajność:** Czas odpowiedzi API dla standardowych operacji wynosi średnio poniżej 100ms (w środowisku lokalnym).
- **Skuteczność OCR:** Moduł rozpoznawania tekstu poprawnie odczytuje kwoty z paragonów w około 80
- **Użyteczność:** Testy z udziałem użytkowników potwierdziły, że interfejs jest przejrzysty, a proces zakładania domu i zapraszania domowników nie sprawia trudności.

System jest gotowy do wdrożenia w wersji produkcyjnej (MVP).

6. Podsumowanie i wnioski

Celem niniejszej pracy inżynierskiej było zaprojektowanie i zaimplementowanie systemu wspomagającego zarządzanie gospodarstwem domowym. Osiągnięto to poprzez stworzenie aplikacji mobilnej „Household Manager” oraz dedykowanego serwera API.

6.1 Realizacja celów pracy

Wszystkie główne założenia projektowe zostały zrealizowane. Powstała aplikacja umożliwia:

- Tworzenie grup użytkowników (domów) i zarządzanie ich członkami.
- Ewidencjonowanie i przydzielanie zadań domowych.
- Transparentne zarządzanie finansami i rozliczanie wspólnych wydatków.
- Automatyzację wprowadzania danych z rachunków dzięki technologii OCR.
- Wspólne planowanie zakupów i demokratyczne podejmowanie decyzji poprzez ankiety.

Wykorzystanie nowoczesnych technologii, takich jak język Go na backendzie oraz React Native na frontendzie, pozwoliło na stworzenie wydajnego i skalowalnego rozwiązania, działającego na dwóch najpopularniejszych platformach mobilnych (Android i iOS).

6.2 Wnioski z realizacji projektu

Podczas realizacji projektu wyciągnięto następujące wnioski:

- Architektura mikroserwisowa (lub modularny monolit w kontenerach Docker) znacząco ułatwia wdrażanie i skalowanie aplikacji.
- Biblioteka GORM znacznie przyspiesza pracę z bazą danych, jednak w przypadku bardzo skomplikowanych zapytań SQL konieczna jest znajomość natywnego języka zapytań.
- Technologia Expo w ekosystemie React Native jest dojrzałym rozwiązaniem, które pozwala na szybkie prototypowanie i budowanie produkcyjnych aplikacji bez konieczności głębokiej ingerencji w kod natywny (Java/Swift).
- Integracja z systemami OCR jest wyzwaniem ze względu na różnorodność formatów paragonów i jakość zdjęć. Wymaga to implementacji mechanizmów walidacji i korekty danych przez użytkownika.

6.3 Kierunki dalszego rozwoju

System posiada potencjał do dalszej rozbudowy. Możliwe kierunki rozwoju to:

1. **Wersja przeglądarkowa (Web):** Stworzenie panelu administracyjnego dostępnego przez przeglądarkę internetową.
2. **Integracja z bankowością (Open Banking):** Automatyczne pobieranie transakcji z kont bankowych użytkowników w celu łatwiejszego rozliczania.
3. **Rozbudowa modułu AI:** Wykorzystanie bardziej zaawansowanych modeli uczenia maszynowego do lepszego kategoryzowania wydatków i inteligentnego przydzielania zadań (np. na podstawie preferencji i historii).
4. **Kalendarz:** Wizualizacja zadań i płatności na wspólnym kalendarzu z możliwością synchronizacji z Google Calendar.

Podsumowując, aplikacja „Household Manager” stanowi funkcjonalne i nowoczesne narzędzie, które realnie może wpłynąć na poprawę jakości życia osób dzielących gospodarstwo domowe, redukując liczbę konfliktów i usprawniając codzienne obowiązki.

Spis literatury

Książki

Artykuły

Źródła internetowe i inne

A. Instrukcja instalacji i uruchomienia

W tym dodatku opisano procedurę instalacji i uruchomienia systemu „Household Manager” w środowisku lokalnym.

A.1 Wymagania wstępne

Do poprawnego uruchomienia systemu wymagane jest zainstalowanie następujących narzędzi:

- **Docker Desktop** (wersja 4.0 lub nowsza) – do konteneryzacji serwera i bazy danych.
- **Node.js** (wersja 18 LTS lub nowsza) – środowisko uruchomieniowe dla aplikacji mobilnej.
- **Git** – system kontroli wersji.
- **Aplikacja Expo Go** – zainstalowana na fizycznym urządzeniu mobilnym (Android/iOS) do testowania aplikacji.

A.2 Uruchomienie serwera (Backend)

1. Sklonuj repozytorium projektu:

```
1 git clone https://github.com/Dragodui/diploma-server.git
2 cd diploma-server
```

2. Utwórz plik konfiguracyjny `.env` na podstawie przykładu:

```
1 cp .env.example .env
```

3. Uzupełnij brakujące zmienne środowiskowe w pliku `.env` (klucze API do AWS S3, Google OAuth itp.). Dla celów testowych lokalnie można pozostawić wartości domyślne dla bazy danych.

4. Uruchom kontenery Docker:

```
1 docker compose up -d --build
```

5. Serwer będzie dostępny pod adresem `http://localhost:8080`. Dokumentacja Swagger UI dostępna jest pod adresem `http://localhost:8080/swagger/index.html`.

A.3 Uruchomienie aplikacji mobilnej (Frontend)

1. Przejdź do katalogu aplikacji klienckiej:

```
1 cd client
```

2. Zainstaluj zależności projektu:

```
npm install
```

3. Uruchom serwer deweloperski Expo:

```
npx expo start
```

4. Zeskanuj wygenerowany kod QR za pomocą aplikacji Expo Go na swoim telefonie (upewnij się, że telefon i komputer są w tej samej sieci Wi-Fi) lub naciśnij a, aby uruchomić emulator Androida (wymaga zainstalowanego Android Studio).

A.4 Konfiguracja zmiennych środowiskowych

Poniżej przedstawiono przykładową konfigurację pliku .env dla serwera:

```
PORT=8080
DB_HOST=postgres
DB_USER=postgres
DB_PASSWORD=secret
DB_NAME=household_db
JWT_SECRET=super_secret_key_change_me
AWS_REGION=eu-central-1
AWS_ACCESS_KEY_ID=AKIA...
AWS_SECRET_ACCESS_KEY=...
```

Algorytm A.1: Przykładowy plik .env